FERHAOUI Yanisse 11909519

Rapport TP OpenGL

Le code a été réalisé en plusieurs étapes :

1. Le terrain et les cubes

Le terrain a été chargé de manière très naïve, à l'aide du fichier Clipboard01.png, les cubes utilisés proviennent du pack « Cube World » trouvé sur Quaternius. Les cubes sont ensuite chargés dans un tableau de *buffers*, dans l'objectif de dessiner un cube selon son indice (et donc selon la hauteur de la carte au point (x,y)).

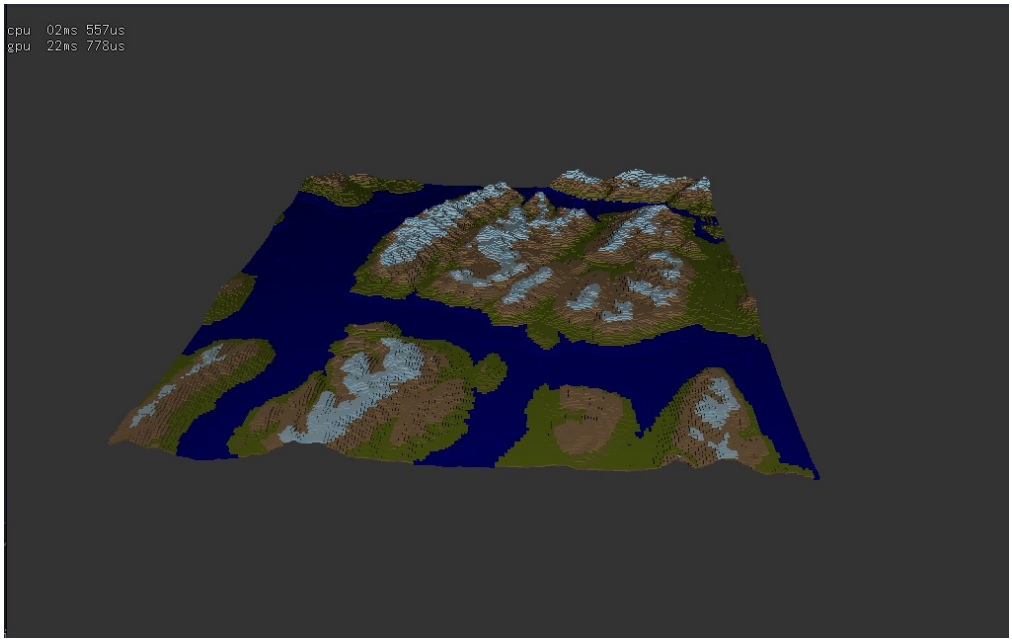
a) Dessiner les cubes : première méthode

Pour dessiner les cubes, cela se passe en bonne partie dans la fonction **render**, une double boucle parcourant l'image de terrain récupère la hauteur, effectue les bonnes translations du modèle en x, y et z, attribue le bon index de cube à dessiner en fonction de la hauteur, et fait appel à **glDrawElement** pour dessiner les cubes 1 par 1. L'image étant de taille 243x243, il est évident que le temps de calcul est très impactée, sur CPU mais majoritairement sur GPU. L'objectif suivant est donc de pouvoir réduire le nombre d'appels à la fonction draw.

b) Dessiner les cubes : deuxième méthode

Pour régler le problème rencontré dans la partie a), la solution envisagée est d'utiliser la fonction **glDrawElementInstanced**, qui permet de faire de l'instanciation. L'instanciation permet de dessiner un même objet plusieurs fois en un seul appel. Quels changements ont dû être fait au niveau du code pour que l'on puisse instancier les appels à draw? Il a fallu ajouter dans la structure *buffer* les matrices d'instances, qui sont réalité la transposée de la matrice model mais cette fois-ci envoyées dans le shader directement dans la fonction init, et non render. Dans le render, les matrices projection et vue sont également envoyés sur GPU pour calculer la matrice MVP. Et enfin le **glDrawElementInstanced** est exécuté une fois par cube. Cela a en effet diminué drastiquement le temps de calcul.

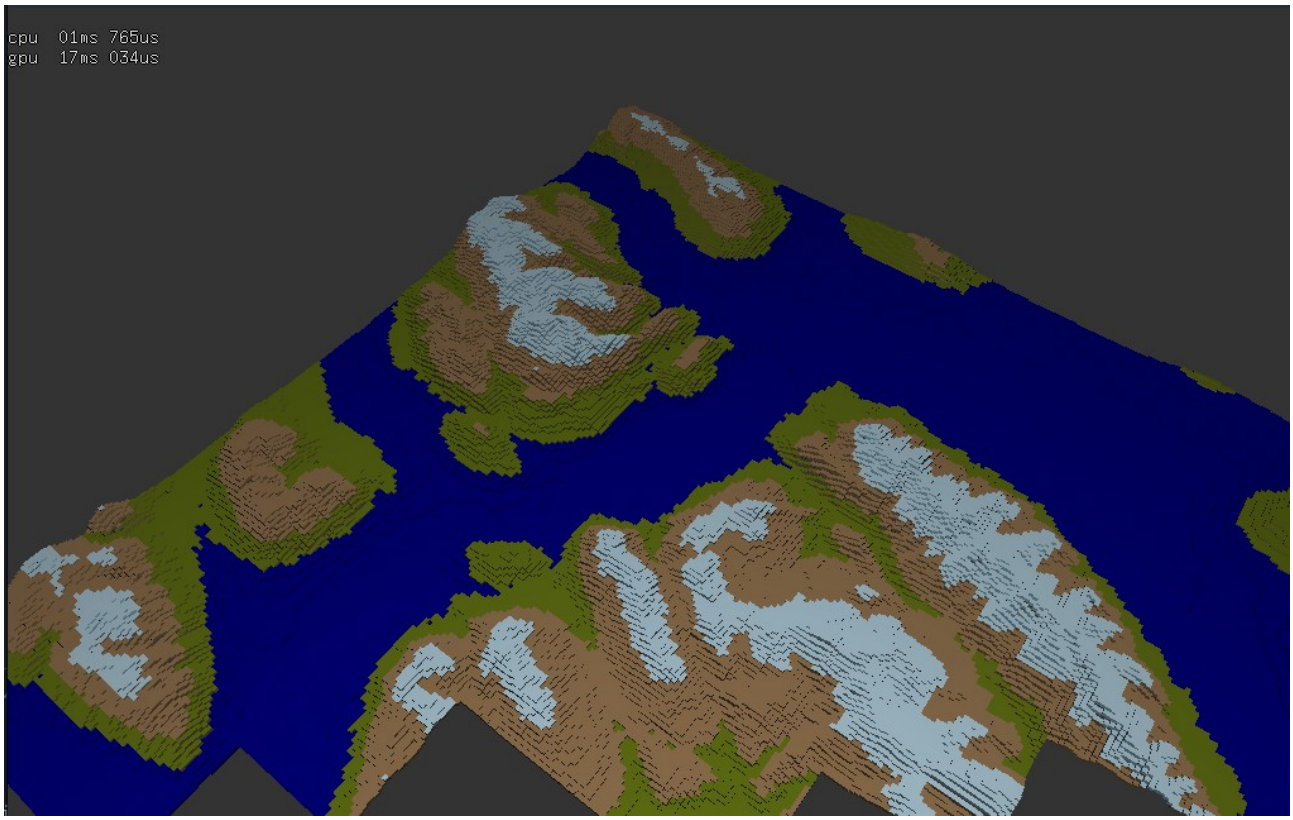
La carte obtenue est la suivante :



2. Frustum culling

Pour rendre le calcul encore plus efficace, on pouvait également faire en sorte de ne dessiner que les cubes étant dans le frustum de la caméra. Le frustum de la caméra étant la région du repère monde pouvant être vu à l'écran. Pour se faire, les structures *Plane*, *AABB* et *Region* ont été créé, une fonction **initializeRegions** permettant de découper la carte en un nombre de régions passé en paramètres, une fonction **extractFrustumPlanes** permettant de récupérer les différents plans du frustum et de vérifier s'ils contiennent la boîte englobante des régions avec **isAABBVisible**. Le buffer est ensuite mis à jour dans la fonction **cullRegionsAndUpdateBuffers**.

Avec 32 régions nous obtenons ceci :

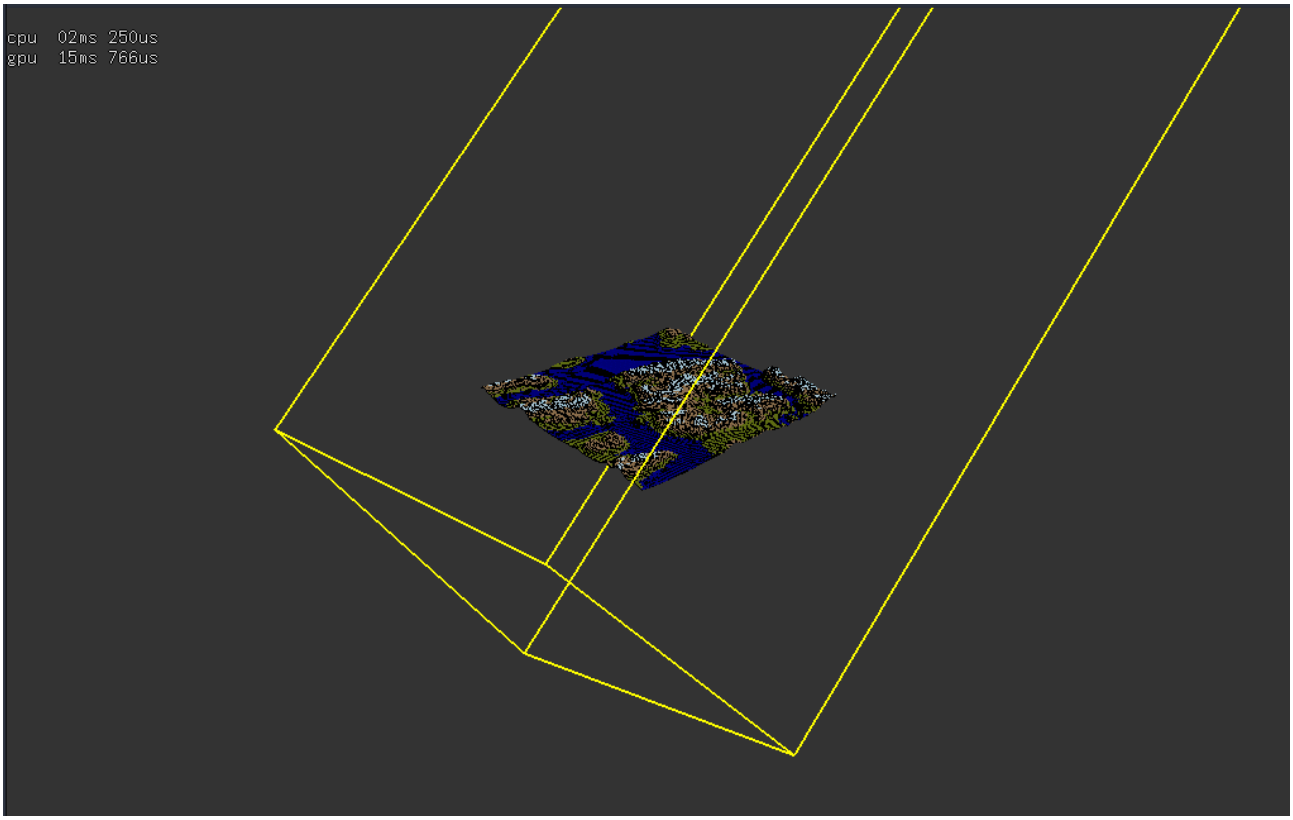


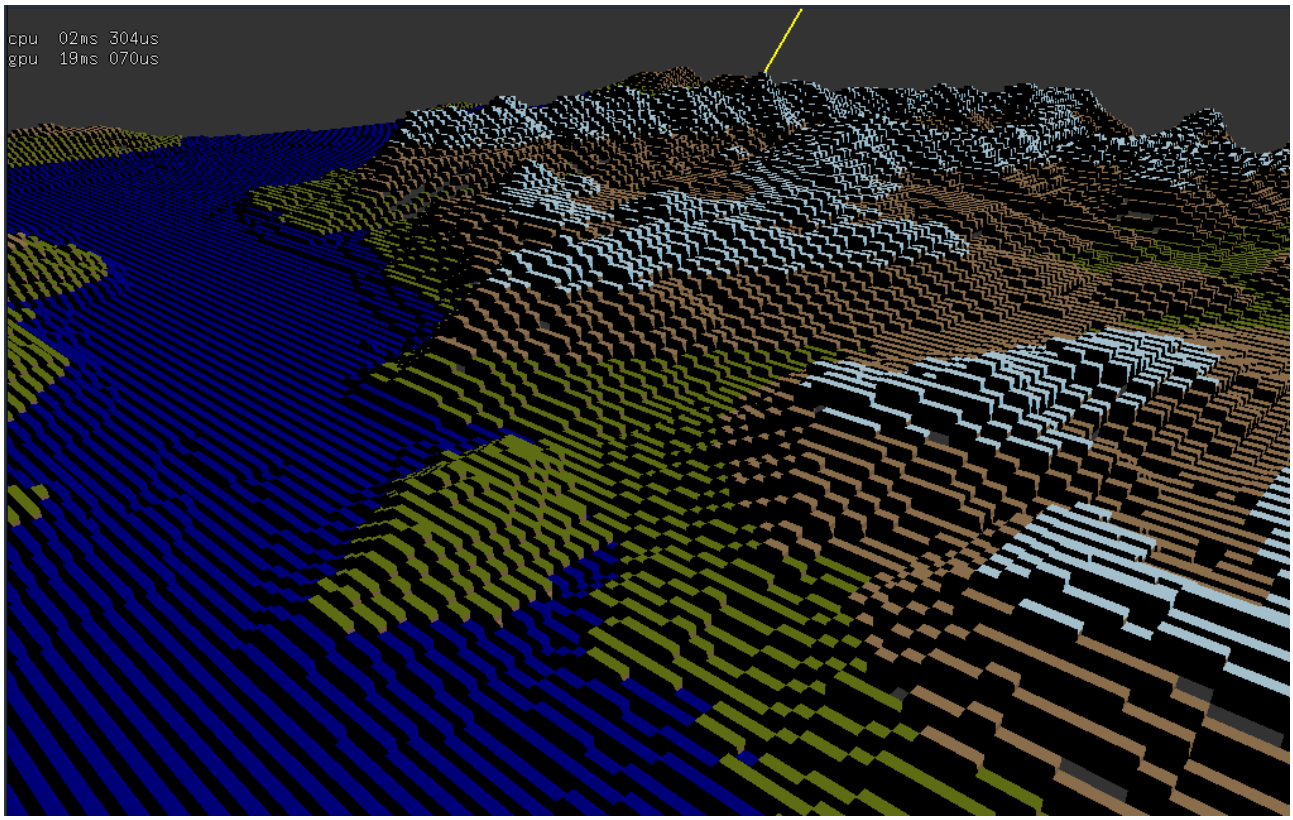
Au Sud de l'image, les régions n'étant pas majoritairement dans le frustum de la caméra sont masqué, et un léger gain de performance est visible.

3. Framebuffer et shadow map

La dernière étape de ce projet est de dessiner les ombres. Pour se faire, il a fallu ajouter un framebuffer, pour que l'on puisse dessiner dedans la shadow map dans un premier temps, puis le terrain dans un 2e temps. Dans la fonction render, la matrice représentant la direction de la lumière est envoyée dans le shader. Dans ce dernier, une fonction de calcul d'ombre est créée dans le fragment shader et permet aux morceaux de cubes non touchés par la lumière d'être noirs.

La projection de l'ombre d'un cube sur ceux d'en dessous n'a pas fonctionné, voici le résultat final (en jaune la projection de la matrice lumière) :





Le code est éxécutable sur gKit, il vient initialement de tuto7_camera.cpp et de tuto7_camera.glsl